

Pygame for Fly-in Guide v2

Oct 6, 2026 · @Ladook

1. How pygame thinks

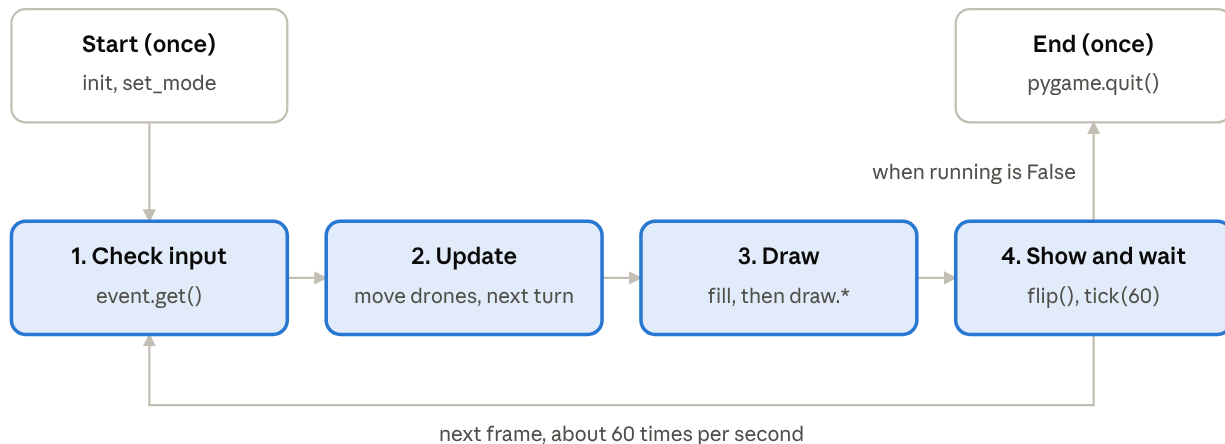
Pygame works like a **flipbook**. A flipbook is many still drawings; flip the pages fast and the drawings seem to move.

In pygame, one still drawing is called a **frame**. Your program makes about 60 frames every second. Nothing on screen ever "moves" by itself. To move a drone, you draw the whole picture again with the drone a little further along.

So every frame, your program does the same four jobs, over and over, until the window closes:

1. **Check input** — did the user press a key or click the close button?
2. **Update** — change your data (for example, a drone's position).
3. **Draw** — erase the old picture and draw the new one.
4. **Show and wait** — put the new picture on screen, then wait a moment so the next frame starts on time.

Every frame runs the same four steps, then starts again



the game loop · 4 steps per frame

Start runs once, the four tinted steps repeat every frame, and End runs once when the loop stops.

This repeating loop is called the **game loop**. Almost every pygame program is built around it.

One more idea you need: pygame has two canvases.

- A **hidden canvas** — this is where you draw. The user can't see it.
- A **visible canvas** — this is what's on the monitor.

You draw the whole frame on the hidden one, then swap them. This way the user never sees a half-finished drawing. Section 3 shows the function that does the swap (`flip`).

2. Setup

Install pygame with pip, using the same Python that runs your `main.py` :

```
python3 -m pip install pygame
```

(On the school cluster, if it says permission denied, add `--user` at the end.)

Check that it installed:

```
python3 -c "import pygame; print(pygame.version.ver)"
```

If you see a version number like `2.6.1` , it works. Also add the line `pygame` to your `requirements.txt` , so whoever evaluates your project can install it.

3. Your first program, line by line

This program opens a black window that you can close. Run it first, then read each part below.

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Fly-in")
clock = pygame.time.Clock()

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.fill((0, 0, 0))
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

`pygame.init()`

- **What it does:** turns on all the parts of pygame (window, fonts, keyboard, sound).
- **Why the name:** short for *initialize* = "prepare to start."
- **If you forget it:** some things break later, for example fonts give the error `font not initialized`.

`screen = pygame.display.set_mode((800, 600))`

- **What it does:** opens the window, 800 pixels wide and 600 pixels tall. It gives back the **hidden canvas** from section 1. We store it in `screen`, and everything you draw goes onto `screen`.
- **Why the name:** `display` is the part of pygame that handles the window. "Mode" is an old word for screen settings, like resolution.
- **Note:** the size is one value, a tuple `(width, height)`, so there are double brackets.
- **About `screen`:** pygame calls a canvas a **Surface** (a flat thing you draw on). `screen` is a Surface. Text images (section 4) are Surfaces too.

`pygame.display.set_caption("Fly-in")`

- **What it does:** sets the title text in the window's top bar.

- **Why the name:** a *caption* is a short title.

```
clock = pygame.time.Clock()
```

- **What it does:** makes a clock object. You use it at the end of each frame (`clock.tick()`) to control speed.

```
running = True and while running:
```

- This is plain Python, not pygame. `running` is your own variable. The loop repeats until something sets it to `False`. Each pass of this loop = one frame.

```
for event in pygame.event.get():
```

- **What it does:** an **event** is something that happened: a key press, a mouse click, the close button. Pygame saves them in a waiting line (a *queue*). `get()` takes them all out and gives you a list, which you loop over.
- **Why it matters:** you must call this every frame. If you don't, your computer thinks the program froze and shows "Not responding."

```
if event.type == pygame.QUIT:
```

- **What it does:** every event has a `type`, which says what kind it is. `pygame.QUIT` is the type for "the user clicked the X button."
- **Then:** we set `running = False`, so the loop ends after this frame.
- **If you forget it:** clicking X does nothing.

```
screen.fill((0, 0, 0))
```

- **What it does:** paints the whole canvas one color, like the paint bucket in Paint. `(0, 0, 0)` is black (colors are explained in section 4).
- **Why it's needed:** it **erases the last frame**. Pygame doesn't clear anything for you.
- **If you forget it:** old drawings stay. A moving drone leaves a smeared trail behind it.

```
pygame.display.flip()
```

- **What it does:** swaps the hidden canvas and the visible one. What you just drew appears on the monitor all at once.
- **Why the name:** like flipping a card over: the back side (where you drew) becomes the front side (what you see).

- **If you forget it:** you draw, but the screen never shows it. The window stays black.
- **Rule:** call it **once per frame, after all drawing**.

```
clock.tick(60)
```

- **What it does:** waits just long enough that the loop runs at most **60 times per second** (60 FPS, *frames per second*).
- **Why the name:** one *tick* of the clock per frame.
- **Bonus:** it returns how many milliseconds passed since the last tick. You'll use this for animation in section 7.
- **If you forget it:** the loop runs as fast as your computer can. It uses 100% CPU, and animations run at different speeds on different computers.

```
pygame.quit()
```

- **What it does:** the opposite of `init()`. Shuts pygame down and closes the window.
- **Where:** once, after the loop, not inside it.

4. Drawing: colors, shapes, text

All drawing goes **between** `screen.fill(...)` and `pygame.display.flip()`. Things drawn later appear on top of things drawn earlier.

Pixels and positions

The window is a grid of tiny dots called **pixels**. An 800 × 600 window has 800 pixels across and 600 down.

A position on screen is `(x, y)`:

- `x` = how many pixels from the **left** edge.
- `y` = how many pixels from the **top** edge.

So `(0, 0)` is the top-left corner, and `(799, 599)` is the last pixel at the bottom-right (counting starts at 0). Careful: **y grows downward**, unlike math class. Section 5 explains how to turn your map coordinates into these.

Colors

A color is three numbers, (red, green, blue), each from 0 to 255. They say how much red, green and blue light to mix.

Color	Value
black	(0, 0, 0) — no light
white	(255, 255, 255) — all light
red	(255, 0, 0)
green	(0, 255, 0)
grey	(128, 128, 128)

You can also use a name: `pygame.Color("green")`. Your map files use color names, so this is handy. If the name is unknown to pygame, it raises `ValueError`, so catch it and use a default color.

```
pygame.draw.circle(screen, color, (x, y), radius, width)
```

Draws a circle.

- `screen` — which canvas to draw on.
- `color` — the color.
- `(x, y)` — the **center** of the circle, in pixels.
- `radius` — distance from center to edge, in pixels.
- `width` — optional. Leave it out (or `0`) for a filled circle. A number like `3` draws only an outline, 3 pixels thick.

For Fly-in: one circle per hub.

```
pygame.draw.line(screen, color, (x1, y1), (x2, y2), width)
```

Draws a straight line from point 1 to point 2. `width` = thickness in pixels.

For Fly-in: one line per connection, from one hub's center to the other's.

```
pygame.draw.rect(screen, color, rect, width)
```

Draws a rectangle. `rect` is made with `pygame.Rect(x, y, w, h)`:

- `x, y` — the **top-left** corner.
- `w, h` — width and height in pixels.

For Fly-in: useful for a box behind the info text.

Text: three steps

Pygame can't draw text directly. It first turns text into a small picture, then pastes the picture onto the screen.

Step 1, once, before the loop — make a font:

```
font = pygame.font.SysFont(None, 20)
```

- `SysFont` = a font from your system (computer).
- `None` = use pygame's default font.
- `20` = text size in pixels.
- Do this **once**. Making a font every frame is slow.

Step 2, in the loop — turn text into a picture:

```
label = font.render("start", True, (255, 255, 255))
```

- `render` = "make a picture of."
- `"start"` — the text.
- `True` — *antialias*: smooth the letter edges. Always use `True`.
- The color.
- `label` is now a small Surface (a picture) holding the word.

Step 3, in the loop — paste it on the screen:

```
screen.blit(label, (x, y))
```

- `blit` means "copy this picture onto that canvas." The name comes from *block image transfer*, an old graphics term.
- `(x, y)` is where the label's **top-left** corner goes.
- To center a label under a hub, use `label.get_width()` (the picture's width in pixels): start at `x - label.get_width() / 2`.

5. Map coordinates to screen pixels

Your map and pygame use two different kinds of positions. You need a small conversion between them.

The names used in this section

Name	Meaning	Example
<code>x</code> , <code>y</code>	A hub's position in your map file . Unit: map steps, not pixels. Can be negative.	<code>slow_path1</code> is at <code>(1, -1)</code>
<code>px</code> , <code>py</code>	The same hub's position on the screen , in pixels. The <i>p</i> stands for <i>pixel</i> . This is what pygame needs.	<code>(180, 60)</code>
<code>x_min</code> , <code>y_min</code>	The smallest x and smallest y among all hubs in the map.	<code>0</code> and <code>-1</code>
<code>x_max</code> , <code>y_max</code>	The largest x and largest y among all hubs.	<code>4</code> and <code>0</code>
<code>cell</code>	How many pixels one map step becomes. You choose it. Bigger = bigger map on screen.	<code>120</code>
<code>margin</code>	Empty pixels around the edge, so hubs aren't cut in half by the window border. You choose it.	<code>60</code>

Why a conversion is needed

1. **Size:** map step 1 → 2 is too small as 1 pixel. You need to stretch it (`cell`).
2. **Negative numbers:** your map has `y = -1`. A pixel can't be at -1; it would be outside the window. So you shift everything until the smallest value becomes 0 (subtract `x_min`, `y_min`).
3. **Edges:** a hub at pixel 0 would be cut in half. So you push everything in a bit (`margin`).

The formula

```
px = margin + (x - x_min) * cell
py = margin + (y - y_min) * cell
```

Read it right to left, in three steps:

1. `x - x_min` — **shift**: the leftmost hub becomes 0. Nothing is negative anymore.
2. `* cell` — **stretch**: map steps become pixels.
3. `margin +` — **push in**: away from the window edge.

Window size

The window must be big enough for the whole map plus a margin on both sides:

```
width  = 2 * margin + (x_max - x_min) * cell
height = 2 * margin + (y_max - y_min) * cell
```

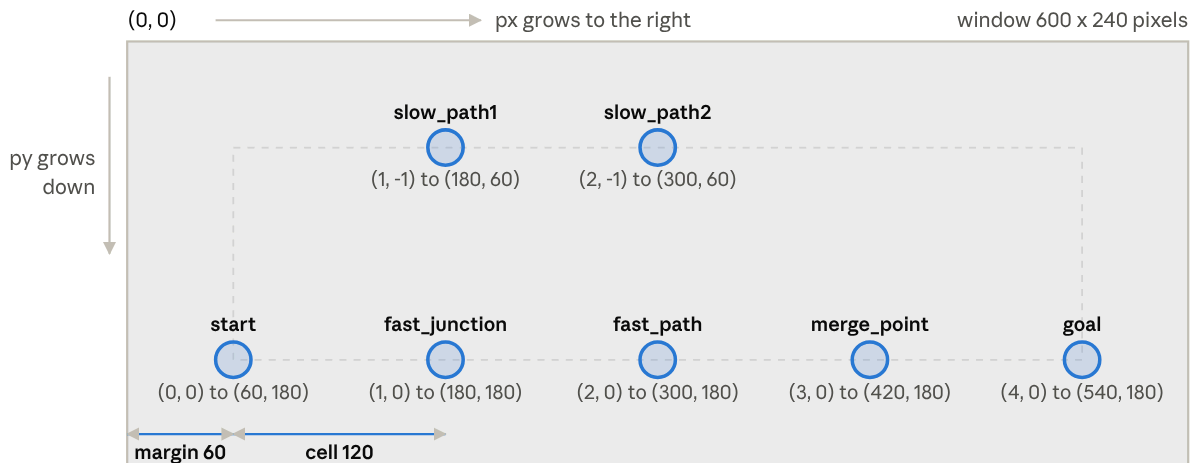
Worked example: 03_priority_puzzle

Hubs from your map: `x` goes from 0 to 4, `y` from -1 to 0. So `x_min = 0`, `x_max = 4`, `y_min = -1`, `y_max = 0`. We choose `cell = 120`, `margin = 60`.

Window: $\text{width} = 2 \times 60 + (4 - 0) \times 120 = \mathbf{600}$. $\text{Height} = 2 \times 60 + (0 - (-1)) \times 120 = \mathbf{240}$.

Hub	Map (x, y)	After shift (x - x_min, y - y_min)	Screen (px, py)
start	(0, 0)	(0, 1)	(60, 180)
slow_path1	(1, -1)	(1, 0)	(180, 60)
slow_path2	(2, -1)	(2, 0)	(300, 60)
fast_junction	(1, 0)	(1, 1)	(180, 180)
fast_path	(2, 0)	(2, 1)	(300, 180)
merge_point	(3, 0)	(3, 1)	(420, 180)
goal	(4, 0)	(4, 1)	(540, 180)

Each hub lands at its (px, py) inside a 600 x 240 window



Each label: map (x, y) to screen (px, py). Dashed box = where hubs can sit, inside the margin.

03_priority_puzzle drawn in pixels · cell 120, margin 60

The same seven hubs as the table, drawn where pygame would put them: the $y = -1$ hubs end up on the top row.

Check one by hand — **slow_path1** at (1, -1) :

- $px = 60 + (1 - 0) \times 120 = 60 + 120 = 180$
- $py = 60 + (-1 - (-1)) \times 120 = 60 + 0 = 60$

One thing to decide: which way is up?

With this formula, $y = -1$ is drawn **above** $y = 0$, because screen y grows downward. If you want it like math class (bigger y = higher on screen), flip y:

```
py = margin + (y_max - y) * cell
```

Both are correct; pick one and use it everywhere.

Tip

Put this conversion in **one** function that takes (x, y) and returns (px, py). Every drawing call uses it. Then changing **cell** resizes the whole map in one place.

6. Applying it to Fly-in

Keep all pygame code in **its own class**, for example `Visualizer`. It only **reads** your hubs, connections and drones and draws them. It never decides where drones go. That way your simulation works the same with or without a window.

What the Visualizer needs

- In `__init__` (once): `pygame.init()`, compute `x_min`, `x_max`, `y_min`, `y_max` (section 5), open the window with the computed width and height, make the clock, make the font.
- A method to convert `(x, y) → (px, py)` (section 5).
- One method per thing to draw: e.g. draw connections, draw hubs, draw drones, draw info.
- A run method with the game loop (section 3), calling those draw methods between `fill` and `flip`.

What to draw, in order

Later items are drawn on top, so draw lines first, otherwise the lines would cross over your hubs.

Order	What	Function	Data it needs from your objects
1	Clear the screen	<code>screen.fill</code>	—
2	Connections	<code>pygame.draw.line</code>	the two hubs' <code>(px, py)</code>
3	Hubs	<code>pygame.draw.circle</code>	<code>(px, py)</code> , color, zone type
4	Hub labels	<code>font.render</code> + <code>screen.blit</code>	name, drones now / max drones
5	Drones	<code>pygame.draw.circle</code> (small)	each drone's <code>(px, py)</code>
6	Info text	<code>font.render</code> + <code>screen.blit</code>	turn number, drones arrived
7	Show it	<code>pygame.display.flip</code>	—

Showing zone types

Using only the functions from section 4:

- **blocked** — grey hub, plus two crossing `draw.line` calls as an X.
- **restricted** — a thick outline: `draw.circle` with `width = 4`.
- **priority** — a second, smaller circle inside the hub.

Build it step by step

Test after every step; don't write everything at once.

1. A window opens and closes (section 3).
2. Hubs appear in the right places (check against the table in section 5).
3. Connections between the hubs.
4. Labels under the hubs.
5. Drones drawn standing still.
6. Drones moving each turn (section 7).

7. Animation: drones moving each turn

The simplest plan: run your whole simulation first and save the result as a list of **turns**. Each turn says, for every drone, which hub it starts at and which hub it ends at. The Visualizer then just plays this list back, like a video.

The names used in this section

Name	Meaning	Example
<code>turns</code>	List of all turns from your simulation.	6 turns
<code>turn</code>	Which turn is showing now (an index into <code>turns</code>).	<code>2</code>
<code>t</code>	How far through the current turn we are. <code>0.0</code> = at the start hub, <code>1.0</code> = arrived.	<code>0.5</code> = halfway
<code>dt</code>	Seconds since the last frame (d = difference, t = time).	about <code>0.016</code> at 60 FPS

Name	Meaning	Example
<code>seconds_per_turn</code>	How long one turn takes on screen. You choose it.	<code>1.0</code>
<code>start_px,</code> <code>start_py</code>	Pixel position of the hub the drone leaves.	<code>(60, 180)</code>
<code>end_px, end_py</code>	Pixel position of the hub it goes to.	<code>(180, 180)</code>

Step 1: let time pass

```
dt = clock.tick(60) / 1000
t = t + dt / seconds_per_turn
if t >= 1.0:
    t = 0.0
    turn = turn + 1
```

- `clock.tick(60)` returns milliseconds since last frame. `/ 1000` turns it into seconds.
- `dt / seconds_per_turn` = what fraction of the turn passed this frame. With `seconds_per_turn = 1.0`, after 60 frames (1 second) `t` reaches 1.0.
- When `t` reaches 1.0, the turn is done: go to the next turn and start from 0 again.
- Why use `dt` instead of adding a fixed number? So drones move at the same speed on a fast or slow computer.
- This replaces the plain `clock.tick(60)` from section 3; call `tick` only once per frame.
- Stop when `turn` reaches `len(turns)`, so you don't read past the end of the list.

Step 2: where is the drone right now?

```
px = start_px + (end_px - start_px) * t
py = start_py + (end_py - start_py) * t
```

This is called **lerp** (*linear interpolation*): find a point on the straight line between two points.

- `end_px - start_px` = the full distance to travel.
- `* t` = the part of it already traveled.
- `start_px +` = start from the start hub.

Example: from `start (60, 180)` to `fast_junction (180, 180)`.

t	px	Drone is...
0.0	$60 + 120 \times 0.0 = 60$	on <code>start</code>
0.5	$60 + 120 \times 0.5 = 120$	halfway
1.0	$60 + 120 \times 1.0 = 180$	on <code>fast_junction</code>

(`py` stays 180 because both hubs have the same `py`.)

A drone that **waits** this turn has `start = end`, so the formula keeps it in place. No special case needed.

Step 3: keyboard control

Add these checks inside the same `for event in pygame.event.get():` loop as the `QUIT` check:

```
if event.type == pygame.KEYDOWN:
    if event.key == pygame.K_SPACE:
        paused = not paused
    elif event.key == pygame.K_RIGHT:
        turn = turn + 1
```

- `pygame.KEYDOWN` — the event type for "a key was pressed down."
- `event.key` — which key it was.
- `pygame.K_SPACE` , `pygame.K_RIGHT` — names for the space bar and the right arrow (`K` = key).
- `paused` — your own `True / False` variable. When `True`, skip the `t = t + ...` line but keep drawing, so the picture freezes but the window still responds.

8. Quick reference

Function	What it does	When
<code>pygame.init()</code>	Turns pygame on.	Once, at the start

Function	What it does	When
<code>pygame.display.set_mode((w, h))</code>	Opens a $w \times h$ pixel window; returns the canvas (<code>screen</code>).	Once
<code>pygame.display.set_caption(text)</code>	Sets the window title.	Once
<code>pygame.time.Clock()</code>	Makes a clock for frame timing.	Once
<code>pygame.font.SysFont(None, size)</code>	Makes a font, size in pixels.	Once
<code>pygame.event.get()</code>	Takes out all events (keys, clicks, close) since last frame.	Every frame
<code>pygame.QUIT</code>	Event type: X button clicked.	In event loop
<code>pygame.KEYDOWN</code> / <code>event.key</code>	Event type: key pressed / which key.	In event loop
<code>pygame.K_SPACE</code> , <code>K_RIGHT</code> , <code>K_ESCAPE</code>	Names of keys.	In event loop
<code>screen.fill(color)</code>	Paints the whole canvas one color = erases the last frame.	Every frame, first
<code>pygame.draw.line(screen, color, p1, p2, width)</code>	Line from p1 to p2.	Every frame
<code>pygame.draw.circle(screen, color, center, radius, width)</code>	Circle; <code>width > 0</code> = outline only.	Every frame
<code>pygame.draw.rect(screen, color, pygame.Rect(x, y, w, h), width)</code>	Rectangle, (x, y) = top-left.	Every frame
<code>pygame.Color("name")</code>	Color from a name; <code>ValueError</code> if unknown.	Any time
<code>font.render(text, True, color)</code>	Turns text into a picture (Surface).	Every frame
<code>label.get_width()</code>	Width of that picture in pixels (for centering).	Every frame

Function	What it does	When
<code>screen.blit(picture, (x, y))</code>	Pastes a picture onto the canvas, (x, y) = top-left.	Every frame
<code>pygame.display.flip()</code>	Swaps hidden and visible canvas = shows the frame.	Every frame, after drawing
<code>clock.tick(fps)</code>	Waits to keep fps; returns ms since last call.	Every frame, last
<code>pygame.quit()</code>	Turns pygame off, closes the window.	Once, after the loop

Official docs, if you want more: [pygame.org/docs](https://www.pygame.org/docs)